International Journal of Software Engineering and Knowledge Engineering
© World Scientific Publishing Company

Context-Encoded Code Change Representation for Automated Commit Message Generation

Thanh Trong Vu, Thanh-Dat Do, and Hieu Dinh Vo*

Faculty of Information Technology, VNU University of Engineering and Technology, Hanoi, Vietnam

{*thanhvu, 20020045, hieuvd*}*@vnu.edu.vn*

Changes in source code are an inevitable part of software development. They are the results of indispensable activities such as fixing bugs or improving functionality. Descriptions for code changes (commit messages) help people better understand the changes. However, due to a lack of motivation and time pressure, writing high-quality commit messages remains reluctantly considered. Several methods have been proposed with the aim of automated commit message generation. However, the existing methods are still limited because they only utilise either the changed code or the changed code combined with surrounding statements. This paper proposes a method to represent code changes by combining the changed code and the unchanged code which have program dependence on the changed code. Specifically, we first create program dependence graphs (PDG) of source code before and after the change. After that, slices related to the changed code from these PDGs are extracted. These slices are then merged to represent the change. This method overcomes the limitations of current representations while improving the performance of 5/6 of state-of-the-art commit message generation methods by up to 15% in METEOR, 14% in ROUGE-L, and 10% in BLEU-4.

Keywords: code changes representation, automated commit message generation, program dependence, program slices.

1. Introduction

Changes in source code are inevitable activities during the life cycle of software applications. Changes are made in order to fix bugs and improve functionality. Along with changes in source code, writing commit messages, which describe the changes, is an essential part of the software development process [1,2]. Good commit messages help the code reviewers and maintainers easily understand the changes and the rationales behind them [3, 4].

Although commit messages bring many benefits, in many cases, they are overlooked. This issue is affirmed by a survey conducted by Tian *et. al.* [4] on 1,600 randomly selected commit messages from five big open-source software projects. The results indicated that up to 44% of the commit messages lacked crucial information regarding the nature of the code changes and the reasons for their implementa-

*Corresponding author

tion. In addition, a survey by Dyer *et. al.* [5] shows that, in 23,000 Java projects on SourceForge, about 10% of commit messages contain simple sentences, 66% of them have only a few words, and up to 14% of the commit messages are left blank. This means that the current commit messages contain insufficient necessary information to understand the code changes. Also, according to Tian *et. al.*, a high-quality commit message should have two complete parts describing what has changed and why that change has occurred.

Recently, several automated commit message generation methods have been proposed. Among these works, several studies rely on pre-defined rules or patterns [6–8]. Some studies apply information retrieval techniques to reuse commit messages for similar source code changes [9–12]. Recently, Seq2Seq-based neural network models have been introduced to understand code changes and create high-quality commit messages [13–18]. Although these approaches show promising results, they still have limitations that originate from the understanding and the representation of the code changes. Some of these methods represent code changes using only the changed code, which includes the added and removed statements. Because statements in changed code usually interact with the statements in unchanged code, it is unlikely to have high-quality commit messages if only changed code is used for message generation. Some other methods try to overcome this limitation by exploiting both the changed code and its surrounding statements. However, a statement positioned right before/after the changed code does not necessarily have a semantic relationship with the changed code. Using surrounding statements which have no semantic relationship with the changed code to generate the commit message should result in a low-quality one. To have high-quality commit messages, in our opinion, we need to consider the changed code and the unchanged code which has semantic relationships with it.

This paper proposes a context-based representation for code changes. The proposed representation aims at providing information about changes in the program through statements that impact or are impacted by the changed code. Specifically, this representation is created by combining changed codes and unchanged codes that have program dependences on the changed code. In addition, the proposed representation method also can be easily integrated with the current generation commit message methods. Our experiments on a dataset with 31,517 commits collected from 160 popular open source projects show that the proposed representation method can improve 5 out of 6 state-of-the-art methods for automated commit message generation. In particular, with the new change representation, the performance of these methods can be improved by up to 15% in METEOR, 14% in ROUGE-L, and 10% in BLEU-4. Generally, our approach helps developers in writing good commit messages, and in a broad view, it helps improve the quality of software applications. We may implement this approach as an extension of an IDE or of a version control application for practical use.

In brief, this paper makes the following contributions:

(1) *Context-Encoded Representation*: A novel context-based code changes representation with substantial information about the changes.
(2) An extensive experimental evaluation showing the advantages of *Context-Encoded Representation*.
(3) A public dataset of 30K+ commits collected from 160 real-world Java projects, which can be used as a benchmark for evaluating related works.

The rest of this paper is organized as follows. Section 2 provides the foundational knowledge about automated commit messages generation including an example illustrating our motivation and approach. Section 3 delineates the steps for the representation of code changes. Section 4 describes our evaluation methodology for the proposed code change representation. Section 5 presents and analyses the experiment results. We also discuss threats to the validity of our work. The related works are in Section 6 before the conclusion.

2. Background and Motivation

This section presents the background knowledge of the automated commit message generation process. Subsequently, an exemplification of code changes accompanied by a corresponding commit message is provided. This example is used to illustrate our motivation and main ideals for the representation of code changes.

2.1. *Automated Commit Message Generation*

Fig. 1 presents the main steps in the process of automated commit message generation. First, for each change, two source code versions exist, including the versions before and after the change. It is obvious that not all parts of source code is related to the change. Therefore, these two source code versions will be processed and transformed to create an intermediate change representation. At this step, existing research may use either only changed code [11, 14, 16, 19] or both changed code and surrounding statements [10, 12, 13, 15, 17, 20].

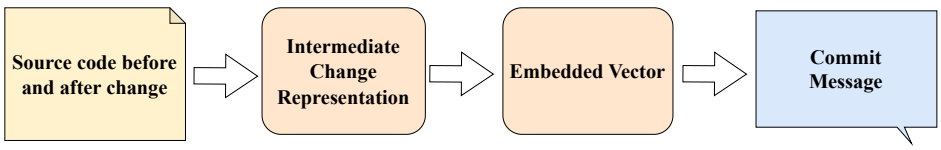

Fig. 1. Typical process for automated commit message generation

Subsequently, this intermediate change representation is transformed into an embedded vector representing the change's semantics. Various embedding approaches have been applied in existing research. Specifically, while Liu *et. al.* utilized the

"Bags of words" model [21] in NNGen [10], recent studies have employed neural networks [11, 13, 15–17] and even pre-trained models [14, 20].

At the last step, commit messages are generated by using information retrieval techniques [10, 11] or, recently, neural networks [13–17, 20]. Given a code change whose commit message needs to be generated, the approaches based on information retrieval techniques manage to find the most similar code change in the database. The generated commit message is the commit message of the most similar code change [10, 11]. Meanwhile, approaches using neural networks [13–17, 20] take the embedded vectors as input for a machine translation process that returns the target commit message.

2.2. *Motivating Example*

Fig. 2 presents an example of a commit message for a simple change in the source code of the open-source project Litho [a]. With this code change, if we only consider the changed code (as the existing studies [11, 14, 16, 19, 20]), what we obtain is the information that the added code is identical to the removed code. In fact, the changed statements may interact with unchanged parts of the program. Therefore, using only the changed code may not fully represent the change and may lead to difficulties in understanding the actual meaning of the change. Moreover, the changed code may be similar between commits, but their purpose and meaning are entirely different because they are combined with different unchanged codes [22]. Therefore, to accurately represent a code change, considering only the changed code seems not enough.

Commit Message: Fixes bug with TextInput TextChangedEvent

File change: litho-widget/src/main/java/com/facebook/litho/widget/TextInputSpec.java

```
@@ -867,13 +867,13 @@ public EditTextWithEventHandlers(Context context) {
   @Override
   protected void onTextChanged(CharSequence text, int start, int lengthBefore, int lengthAfter) {
       super.onTextChanged(text, start, lengthBefore, lengthAfter);
+      if (mTextState != null) {
+          mTextState.set(text);
+      }
       if (mTextChangedEventHandler != null) {
           TextInput.dispatchTextChangedEvent(
             mTextChangedEventHandler, EditTextWithEventHandlers.this, text.toString());
       }
-      if (mTextState != null) {
-          mTextState.set(text);
-      }
       // Line count of changed text.
       int lineCount = getLineCount();
       if (mLineCount != UNMEASURED_LINE_COUNT
```

Fig. 2. An example of a code change and its commit message.

To address the issue of insufficient information about changes when relying

[a] https://github.com/facebook/litho

solely on changed code, people may additionally use the statements surrounding the changed code [10, 12, 13, 15, 17, 20]. Specifically, this approach takes a predefined number n of statements before and after the changed code. In the example, if n is 3, the surrounding statements help to understand that the change moves the block `if (mTextState != null)` from behind to the front of the block `if (mTextChangedEventHandler != null)`. However, surrounding statements also may contain noisy ones. For example, the assignment `int lineCount = getLineCount()` and statement `if (mLineCount != UNMEASURED_LINE_COUNT` have nothing to do with the changed code but still included as its surrounding. These statements may significantly reduce the performance of commit message generation methods.

Therefore, to aim at generating high-quality commit messages, it is necessary to accurately understand the meaning of the change in the source code. This can be achieved by considering the changed code and the unchanged code which is the context of the changed code.

3. Context-Encoded Code Change Representation

In this section, we present the process of building code change representation named *Context-Encoded Representation*, which combines the changed code and statements that are dependent on the changed code. Fig. 3 shows the main steps of our method including the construction of program dependence graphs, program slice extraction, and context-encoded representation construction.

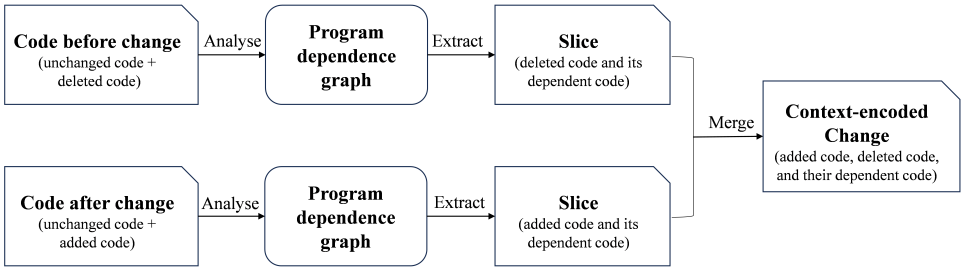

Fig. 3. Main steps in constructing a context-encoded representation for a code change.

3.1. *Program Dependence Graph Construction*

In this study, to represent the dependencies between statements in the source code, we constructed a Program Dependence Graph (PDG). In a PDG, each node represents a statement while edges show relationships between statements. As depicted in Fig. 4, the graph effectively visualizes both data and control relationships between statements. Control dependence exists between two statements if one potentially

prevents the execution of the other. Data dependence occurs when two statements declare, use, or reference the same variable.

For each commit, we build two PDGs: one for the source code before the change (including the deleted code and the unchanged code) and the other for the source code after the change (including the added code and the unchanged code).

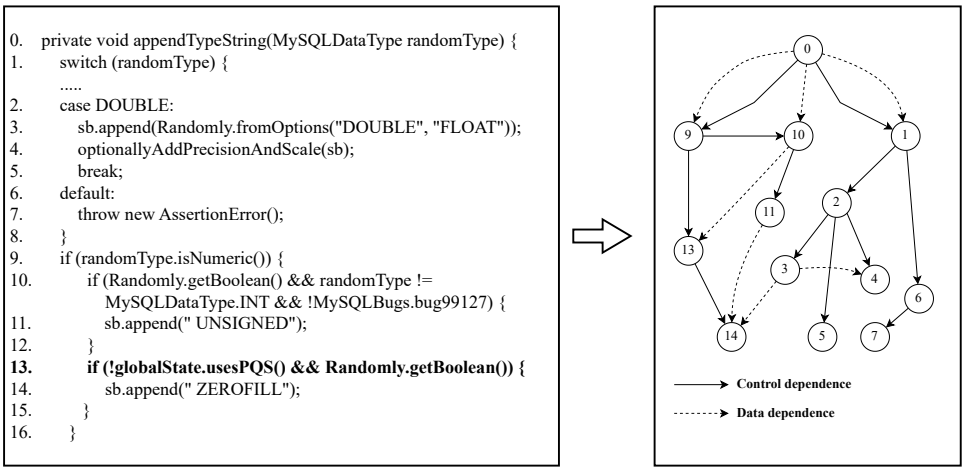

Fig. 4. An example of a program dependence graph

3.2. *Program Slice Extraction*

From the program dependence graph, statements that have program dependences on the changed statements are extracted. Specifically, the proposed method extracts statements that have data dependences and control dependences on the changed code. In particular, we apply both backward and forward slicing. Furthermore, the inter-procedural slicing technique [23] is also applied to ensure that statements outside the function will also be used to represent changes in the source code.

For each commit, with two PDGs created in the previous step, we extract two corresponding slices. The first slice includes the deleted code and its dependent code. Meanwhile, the second slice contains the added code and its dependent code.

3.3. *Context-Encoded Representation Construction*

Fig. 5 illustrates an example of combining program slices of the source code versions before and after the change to build the corresponding context-encoded representation of the change. Specifically, the context-encoded representation is created by combining added statements, removed statements, and unchanged codes that have program dependences on the changed code. In particular, unchanged statements

that are the same between the two program slices will be merged. When combining the before and after versions of a change, the order of statements in the source code is preserved. Furthermore, changed statements are also marked to distinguish them from unchanged codes. Added statements are preceded by a `'+'` character while the corresponding character for removed statements is `'-'`. In particular, context-encoded representations are formatted as a sequence of statements that have program dependences on each other. This format is compatible with the representation of the existing methods for automated commit message generation (but different in content). This allows *Context-Encoded Representation* easily be integrated with the existing methods.

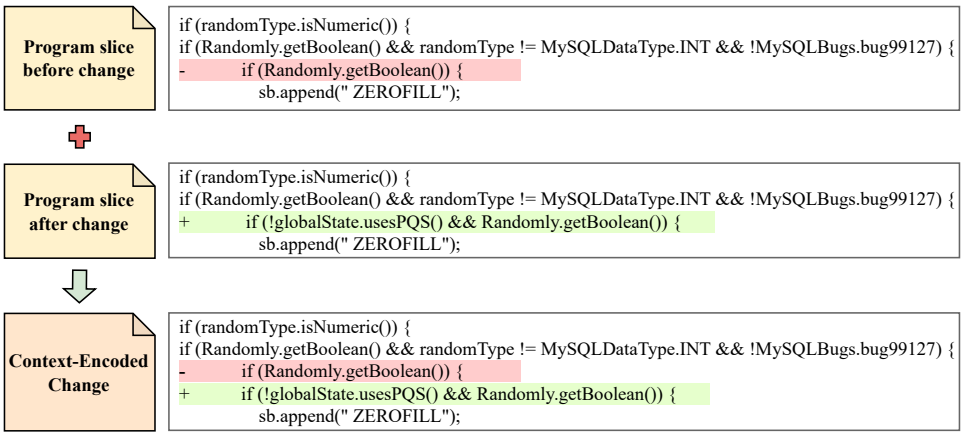

Fig. 5. An example of building *Context-Encoded Representation*

4. Evaluation Methodology

To evaluate our approach, we seek to answer the following research questions:

RQ1: Performance Improvement. How does *Context-Encoded Representation* improve the performance of the state-of-the-art methods for automated commit message generation [10, 11, 14, 20, 24, 25]?

RQ2: Context Extraction Analysis. How important are the dependences in *Context-Encoded Representation* regarding its performance?

RQ3: Dependency Depth Analysis. How does the depth of dependences in *Context-Encoded Representation* impact its performance?

RQ4: Changes Complexity Analysis. How does changed code's complexity affect *Context-Encoded Representation*?

4.1. *Metrics*

The metrics used to evaluate the proposed method include BLEU-4, METEOR, and ROUGE-L, which are commonly used in machine translation researches [12, 16, 19].

BLEU-4: BLEU-4 is calculated based on the n-gram overlap between the generated text and the reference text, with n ranges from 1 to 4. N-gram overlap is determined by dividing the total number of n-grams appearing in both the generated and reference text by the total number of n-grams in the generated text. The value of BLEU-4 is calculated using the formula:

$$BLEU = BP.exp(\sum_{n=1}^{4} w_n log(p_n))$$

where:

- BP is a coefficient reflecting the difference in length between generated and reference text.
- w is a coefficient for different lengths of n-grams.
- p: is the ratio of common n-grams in the generated and reference text.

METEOR: METEOR score is based on unigram *Precision*, unigram *Recall* and the harmonic mean of both. Unigram Precision metrics are the proportion of single words in the generated text that also appear in the reference text. Unigram Recall is the proportion of words in the reference text that also appear in the generated text. The harmonic mean of these two values provides a balance between them. A higher METEOR score indicates better quality generated text, as it is more similar to the reference text.

ROUGE-L: ROUGE-L has commonly used in text summarization, machine translation, and text generation from prompts [12, 16, 19]. It is a recall-oriented metric for evaluating the quality of machine-generated text against a set of reference texts. The ROUGE-L score is calculated by taking the longest common contiguous subsequence between the generated text and the reference text and dividing it by the total length of the reference text. This means that the ROUGE-L score focuses on measuring the coverage of the reference text in the generated text without considering the order of words. A higher ROUGE-L score indicates higher quality generated text.

4.2. *Dataset*

The existing datasets [10] only provide information about the commit message along with the changed code and surrounding statements. This makes these datasets unsuitable for our evaluation. Therefore, we built a dataset by collecting all open-source Java projects which have at least 1,000 stars on Github (160 projects). Commits of these projects were then processed and filtered to meet the following criteria: having a sufficiently large change commit message length, being grammatically

correct, and being well-evaluated by Yingchen's model [4]. Finally, 31,517 quality commits are retained (Table 1).

Table 1. Statistics of the dataset

	#Commit	**#Added line**	**#Removed line**	**#Changed/Commit**
GraalVM	1,763	8,193	4,356	7.12
Bazel	1,493	6,464	4,227	7.16
Buck	1,155	5,425	2,985	7.28
Loom	1,109	4,594	2,939	6.79
Tomcat	1,050	4,545	2,750	6.95
		155 projects more		
Total	31,517	139,776	79,897	6.97

Similar to previously published datasets [10,11,13], for each commit, we only consider the first sentence which usually summarises the content of the whole message. After that, the commit message is further cleansed by removing unique elements such as Issue ID, commit ID, and URL.

In the next step, any commits with too many changed statements will be removed (more than 20 changes). Most of these changes are "merger" or "rollback" commits. In addition, commit messages that are too short (less than five words) are also removed because they are usually meaningless. Messages longer than 150 words are also dismissed because this is the maximum length of commit messages generated by the existing methods. After that, the commit messages are checked for the 'verb direct object' grammatical structure. Finally, they are fed into the deep learning model proposed by Yingchen and colleagues [4] to classify whether they are good or not. Only good commit messages are retained.

4.3. *Experimental Setup*

To evaluate the performance improvement of our method, we select existing state-of-the-art methods for commit message generation. Especially because the existing methods belong to different categories (information retrieval, neural network, and pre-trained), we choose the best methods from each category. Six methods are chosen to include in our experiment including NNGen [10] and CC2Vec [11] (information retrieval), CommitBERT [14] and CommitBART [20] (neural network), and CodeT5 [25] and UniXcoder [24] (pre-trained). Implementations of these methods are provided by their corresponding authors.

To measure the performance of selected methods, we split our dataset into training, validation, and test parts with ratio 70:10:20. The maximum commit message size is 150 words, and the input size for generating commit messages is 512. Pre-trained models used in experiments are CodeT5 and UniXcoder. Specifically for the pre-trained CodeT5 model, batch_size, learning_rate, and the number of epochs are 8, 5e-5, and 10, respectively. The pre-trained model initializing weights are `Salesforce/codet5-base`. For the pre-trained UniXcoder model, batch_size,

learning_rate, and epoch are 12, 5e-5, and 10, respectively. The pre-trained model initializing weights are `microsoft/UniXcoder-base`. For other methods such as CommitBART [20], CommitBERT [14], NNGen [10], and CC2Vec [11], the parameters are set with the default values.

For data collection, we use Pydriller library[b]. Meanwhile, Joern [26] is used for analysing program dependencies. All experiments were performed on a server with an Intel Xeon (2) @ 2.00GHz CPU, 16 GB RAM, and an NVIDIA Tesla P100 PCIe 16GB GPU running Ubuntu 20.04.4 LTS x86_64.

5. Experimental Results

5.1. *Performance Improvement (RQ1)*

To evaluate the impact of *Context-Encoded Representation* on the performance of state-of-the-art methods for automated commit message generation, we experiment with two scenarios: using the existing methods as they are and equipping the existing methods with *Context-Encoded Representation*.

Table 2 describes the performance of automated commit message generation methods in two cases: the original version and the version with *Context-Encoded Representation*. In general, *Context-Encoded Representation* improves 5 out of 6 existing methods. It significantly improves methods using neural networks and pre-trained models. In particular, when applying *Context-Encoded Representation*, for CommitBERT [14], its performance is improved by up to 15% in the METEOR metrics, 14% in ROUGE-L, and 10% in BLEU-4 metrics. Those figures for CommitBART [20] are 5%, 13%, and 9%, respectively. The performance of these methods is improved because when using *Context-Encoded Representation*, neural networks and pre-trained models can grasp the meaning of the change from program-dependent statements.

We can see that *Context-Encoded Representation* does not improve the performance of CC2Vec [11] and NNGen [10]. These methods are based on information retrieval techniques. NNGen uses the "bags of words" model for embedding intermediate change representations.Consequently, this method cannot represent the relationships between words and cannot utilize the program dependences provided by *Context-Encoded Representation*. CC2Vec treats the added and removed source code separately before concatenating their embedded vectors into a single one representing changes in the source code. Therefore, when supplemented with unchanged code from *Context-Encoded Representation*, this method cannot exploit the program dependences. In addition, due to the characteristics of information retrieval techniques, generated commit messages are actually taken from a pre-defined set of messages, and the performance of these methods may reach their upper bounds.

[b]https://pydriller.readthedocs.io

Table 2. The impact of *Context-Encoded Representation* on automated commit message generation

	Methods	METEOR	ROUGE-L	BLEU-4
CommitBERT	Original	4.66	10.81	7.11
	with *Context-Encoded Change*	**5.36**	**12.28**	**7.83**
CommitBART	Original	8.95	15.28	11.11
	with *Context-Encoded Change*	**9.38**	**17.33**	**12.16**
CodeT5	Original	7.71	15.17	9.75
	with *Context-Encoded Change*	**8.46**	**16.29**	**10.54**
UniXcoder	Original	5.45	11.45	7.64
	with *Context-Encoded Change*	**6.01**	**12.24**	**8.44**
NNGen	Original	4.04	9.23	5.83
	with *Context-Encoded Change*	**4.07**	**9.27**	**5.88**
CC2Vec	Original	4.39	10.07	6.21
	with *Context-Encoded Change*	4.31	9.96	6.22

Answer for RQ1: *Context-Encoded Representation* can improve 5/6 state-of-the-art methods for automated commit messages generation and improve performance by up to 15%.

5.2. *Context Analysis (RQ2)*

In this experiment, we analyse the contribution of different code components in *Context-Encoded Representation*Ṡpecifically, we generate commit messages with 04 different scenarios of *Context-Encoded Representation*corresponding to 04 different ways of extracting context: (i) only the changed code, (ii) the changed code combined with its control dependence statements (iii) the changed code combined with its data dependence statements, and (iv) the changed code combined with its program dependence (i.e. both control and data dependence) statements.

Table 3 shows the contributions of different code components on *Context-Encoded Representation*. We can see that *Context-Encoded Representation* is at its best when changed code and program dependences are included. While data dependences are more helpful than control dependences in generating commit messages, we should use both of them (i.e. program dependences) to maximize the benefit from *Context-Encoded Representation*. Specifically, compared with using only changed code, applying both changed code and program dependence helps enhance performance by 8%, 7%, and 10% in BLEU-4, ROUGE-L, and METEOR, respectively.

Answer for RQ2: The dependences within the source code has varying impacts on *Context-Encoded Representation*. The method is at its best when both control dependences and data dependences are used.

Table 3. Performance of *Context-Encoded Representation* with different types of dependence

	METEOR	ROUGE-L	BLEU-4
Changed code	7.71	15.17	9.75
Changed code + control dependence	7.77	15.16	9.80
Changed code + data dependence	8.25	15.97	10.41
Changed code + program dependence	**8.46**	**16.29**	**10.54**

5.3. *Dependence Depth Analysis (RQ3)*

To analyse the impact of the depth of dependences on *Context-Encoded Representation* we equip it with different levels of dependence depth. For example, in Fig. 4, when the depth is 1, the statements that have program dependences with statement 13 are 9, 10, and 14. Meanwhile, when the depth is 2, the statements dependent on the statement 13 are 0, 9, 10, and 14. In our experiment, the depth varies from one to five.

Table 4 describes the impact of dependence depths on generating commit messages. The results show that by increasing the depth of the dependences *Context-Encoded Representation* may provide better performance for generating commit messages. However, *Context-Encoded Representation* is at its best when dependence depth is 3. Above this value, increasing the dependence depth will result in negative impact on commit message generation.

Table 4. Performance of *Context-Encoded Representation* with different context extraction depths

#Deep level	METEOR	ROUGE-L	BLEU-4
Deep 1	8.31	15.97	10.42
Deep 2	8.30	16.00	10.35
Deep 3	8.46	16.29	10.54
Deep 4	8.26	15.95	10.36
Deep 5	7.68	15.08	9.76

Answer for RQ3: The depth of program dependences slightly impacts the performance of *Context-Encoded Representation*.

5.4. *Changes Complexity Analysis (RQ4)*

We also evaluate the impact of complexity in each change on the quality of the corresponding generated commit message. Specifically, we evaluate the quality of generated commit messages for changes including 1 to 5 changed statements, 5 to 10 changed statements, 10 to 15 changed statements, and more than 15 changed statements.

Table 5 shows the impact of different input sizes on *Context-Encoded Representation*. *Context-Encoded Representation* reaches its best performance when the number of changed statements is between 1 and 5. Its performance then gradually

decreases as the number of changed statements increases. The reason is that when the number of changes increases, it is more difficult to grasp the meaning of them fully.

Table 5. Performance of *Context-Encoded Representation* with different input complexity

#Number of changed statements	METEOR	ROUGE-L	BLEU-4
From 1 to 5	8.35	16.05	10.61
From 5 to 10	8.33	16,00	10.18
From 10 to 15	8.20	15.76	10.27
Over 15	8.03	15.14	9.89

Answer for RQ4: The complexity of the change has a significant impact on the performance of *Context-Encoded Representation*. The more complex the change, the lower the performance of *Context-Encoded Representation*.

5.5. *Time Complexity*

In this paper, we use Joern to analyse the program, it takes about 1 second to explore a commit and 0.1 seconds to build the *Context-Encoded Change*. Compared to other representation techniques, there is not much difference in the training and evaluation process when using *Context-Encoded Representation*. With the commit message generation methods using pre-trained models such as CodeT5 [25] or CommitBERT [14], the training time when using *Context-Encoded Representation* and the default method both take around 500 minutes and 540 minutes, respectively.

5.6. *Threats to Validity*

The main threats to the validity of our work consist of internal, construct, and external threats.

Threats to internal validity include the influence of the method used to extract program dependencies. To reduce this threat, we use Joern [26] code analyzer, which is a widely-used code analyzer. Another threat mainly lies in the correctness of the implementation of our approach. To reduce such threats, we carefully reviewed our code and made it public [27] so that other researchers can double-check and reproduce our experiments.

Threats to construct validity relate to the suitability of our evaluation procedure. We used *BLEU-4*, *METEOR*, and *ROUGE-L*. They are the widely-used evaluation measures for evaluating commit messages generated by the existing techniques [13–17, 19]. Besides, a threat may come from the adaptation of the existing commit message generation approaches. To mitigate this threat, we directly obtain the original source code from the GitHub repositories of the studied techniques. Also, we use the same hyperparameters as in the original papers [10,11,14,20,24,25].

Threats to external validity mainly lie in the construction of our dataset. To reduce the impact of this threat, we applied the same data collection procedure as in the existing commit message generation approaches [13–17, 19] and collected the commits in high-quality Java projects. Moreover, our experiments are conducted on only the commits in Java projects. Thus, the results could not be generalized for other programming languages. In our future work, we plan to conduct more experiments to validate our results in other languages.

6. Related Work

Code Change Representation. Our approach is related to code change representation approaches. Before using in a specific task, code changes could be transformed into several forms such as sequence-based [28], tree-based [29], and graph-based [22, 30]. Additionally, several techniques have been proposed to learn how to represent changes [11, 19, 31, 32]. These studies differ from our work as our approach considers the changed code sequences in relation to the related unchanged code and is designed to directly integrate into the existing commit message generation methods.

Commit message generation. Our work is also related to learning-based commit message generation methods. Recently, a number of neural network-based approaches [13–17, 19] have been used to understand the semantics of code changes in the commit and translate them into commit messages. NMTGen [15] and CommitGen [13] treat code changes as pure text and use Seq2Seq neural networks with different attention mechanisms to translate them into commit messages. CoDiSum [16] extracts the structure and semantics of code and creates a multi-layer bidirectional GRU model to better understand the representation of code changes. CommitBERT [14] uses CodeBERT [33], a pre-trained language model for code, to understand the semantics of code changes and applies a transformer-based decoder [34] to generate commit messages. Our study differs from these studies as instead of focusing on end-to-end commit message generation, our method is designed to integrate into these existing commit message generation techniques. Additionally, as shown in Section 5, our approach and those approaches can be applied together to generate better commit messages.

Learning-based SE approaches. Several **learning-based approaches** have been proposed for specific SE tasks including code recommendation/suggestion [35–37], program synthesis [38, 39], static analysis warnings [40, 41], pull request description generation [42, 43], code summarization [44–46], code clones [47], fuzz testing [48], bug detection [49], and program repair [50, 51].

7. Conclusion

Commit message is very important in the field of software development. Numerous methods for automated commit message generation have been proposed, yielding remarkable outcomes. Nonetheless, the representation of code changes in these

methods does not fully cover information about changes and is often accompanied by noise. Our method for representing code changes exploits the unchanged code that had program dependence on the changed code. We apply PDG and slicing to extract only statements that are relevant to the change. Experiment results on a dataset collected from 160 open-source Java projects on Github, including 31,517 commits, show that by using the proposed representation method, the performance of 5 out of 6 state-of-the-art automated commit message generation methods can be improved. In particular, the performance of these methods can be improved by up to 15% METEOR compared to using current representations for code changes. Our dataset is published for future studies on this topic [27]. To help developers write high quality commit messages, which is an essential part of high quality software, our approach may be implemented as an extension of an IDE or of version control software. In the future, to better represent code changes, we may consider types of code change and contextual information about the changes.

References

[1] T. Girba, A. Kuhn, M. Seeberger and S. Ducasse, How developers drive software evolution, in *Eighth international workshop on principles of software evolution (IW-PSE'05)*, IEEE, (Switzerland, 2005), pp. 113–122.

[2] A. E. Hassan, The road ahead for mining software repositories, in *2008 frontiers of software maintenance*, IEEE, (Canada, 2008), pp. 48–57.

[3] L. Pascarella, D. Spadini, F. Palomba, M. Bruntink and A. Bacchelli, Information needs in contemporary code review, *Proceedings of the ACM on Human-Computer Interaction* **2**(CSCW) (2018) 1–27.

[4] Y. Tian, Y. Zhang, K.-J. Stol, L. Jiang and H. Liu, What makes a good commit message?, in *Proceedings of the 44th International Conference on Software Engineering*, (Beijing, China, 2022), pp. 2389–2401.

[5] R. Dyer, H. Nguyen, H. Rajan and N. Tien, Boa: A language and infrastructure for analyzing ultra-large-scale software repositories *Proceedings - International Conference on Software Engineering* , (USA, 2013), pp. 422–431.

[6] L. F. Cortés-Coy, M. Linares-Vásquez, J. Aponte and D. Poshyvanyk, On automatically generating commit messages via summarization of source code changes (2014) 275–284.

[7] J. Shen, X. Sun, L. Bin, H. Yang and J. Hu, On automatic summarization of what and why information in source code changes (06 2016) 103–112.

[8] Y. Huang, Q. Zheng, X. Chen, Y. Xiong, Z. Liu and X. Luo, Mining version control system for automatically generating commit comment (2017) 414–423.

[9] Y. Huang, N. Jia, H.-J. Zhou, X. Chen, Z. Zheng and M. Tang, Learning human-written commit messages to document code changes, *Journal of Computer Science and Technology* **35** (2020) 1258 – 1277.

[10] Z. Liu, X. Xia, A. E. Hassan, D. Lo, Z. Xing and X. Wang, Neural-machine-translation-based commit message generation: How far are we? (2018) 373–384.

[11] T. Hoang, H. J. Kang, D. Lo and J. Lawall, Cc2vec: Distributed representations of code changes (2020) p. 518–529.

[12] E. Shi, Y. Wang, W. Tao, L. Du, H. Zhang, S. Han, D. Zhang and H. Sun, Race: Retrieval-augmented commit message generation (2022) 5520–5530.

[13] S. Jiang, A. Armaly and C. McMillan, Automatically generating commit messages

from diffs using neural machine translation (08 2017).
[14] T. Jung, Commitbert: Commit message generation using pre-trained programming language model (01 2021) 26–33.
[15] P. Loyola, E. Marrese-Taylor and Y. Matsuo, A neural architecture for generating natural language descriptions from source code changes (July 2017) 287–292.
[16] S. Xu, Y. Yao, F. Xu, T. Gu, H. Tong and J. Lu, Commit message generation for source code changes (08 2019) 3975–3981.
[17] Q. Liu, Z. Liu, H. Zhu, H. Fan, B. Du and Y. Qian, Generating commit messages from diffs using pointer-generator network (2019) 299–309.
[18] E. Shia, Y. Wangb, L. Du, H. Zhang, S. Han, D. Zhang and H. Sun, Ecmg: Exemplar-based commit message generation, *arXiv preprint arXiv:2203.02700* (2022).
[19] J. Dong, Y. Lou, Q. Zhu, Z. Sun, Z. Li, W. Zhang and D. Hao, Fira: fine-grained graph-based code change representation for automated commit message generation (2022) 970–981.
[20] S. Liu, Y. Li, X. Xie and Y. Liu, Commitbart: A large pre-trained model for github commits (2023).
[21] C. Manning, P. Raghavan and H. Schutze, Introduction to information retrieval (vol. 1). cambridge: Cambridge university press (2008).
[22] S. Nguyen, T.-T. Nguyen, T. T. Vu, T.-D. Do, K.-T. Ngo and H. D. Vo, Code-centric learning-based just-in-time vulnerability detection, *arXiv preprint arXiv:2304.08396* (2023).
[23] S. Horwitz, T. Reps and D. Binkley, Interprocedural slicing using dependence graphs (1988) p. 35–46.
[24] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou and J. Yin, Unixcoder: Unified cross-modal pre-training for code representation (01 2022) 7212–7225.
[25] Y. Wang, W. Wang, S. Joty and S. Hoi, Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation (01 2021) 8696–8708.
[26] F. Yamaguchi, N. Golde, D. Arp and K. Rieck, Modeling and discovering vulnerabilities with code property graphs (2014) 590–604.
[27] https://github.com/thanhtlx/context-encoded-change (2023).
[28] L. Di Grazia, P. Bredl and M. Pradel, Diffsearch: A scalable and precise search engine for code changes, *IEEE Transactions on Software Engineering* (2022).
[29] M. Dilhara, A. Ketkar, N. Sannidhi and D. Dig, Discovering repetitive code changes in python ml systems (2022) 736–748.
[30] H. A. Nguyen, T. N. Nguyen, D. Dig, S. Nguyen, H. Tran and M. Hilton, Graph-based mining of in-the-wild, fine-grained, semantic code change patterns (2019) 819–830.
[31] F. Zhang, B. Chen, Y. Zhao and X. Peng, Slice-based code change representation learning (2023) 319–330.
[32] R. Cabrera Lozoya, A. Baumann, A. Sabetta and M. Bezzi, Commit2vec: Learning distributed representations of code changes, *SN Computer Science* **2** (2021) 1–16.
[33] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang and M. Zhou, Codebert: A pre-trained model for programming and natural languages (01 2020) 1536–1547.
[34] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser and I. Polosukhin, Attention is all you need (2017) p. 6000–6010.
[35] S. Nguyen, H. Phan, T. Le and T. N. Nguyen, Suggesting natural method names to check name consistencies (2020) 1372–1384.
[36] A. Hindle, E. T. Barr, M. Gabel, Z. Su and P. Devanbu, On the naturalness of software, *Communications of the ACM* **59**(5) (2016) 122–131.
[37] M. Allamanis, H. Peng and C. Sutton, A convolutional attention network for extreme

summarization of source code (2016) 2091–2100.
[38] M. Amodio, S. Chaudhuri and T. W. Reps, Neural attribute machines for program generation, *CoRR* (2017).
[39] T. Gvero and V. Kuncak, Synthesizing java expressions from free-form queries (2015) 416–432.
[40] K.-T. Ngo, D.-T. Do, T.-T. Nguyen and H. D. Vo, Ranking warnings of static analysis tools using representation learning (2021) 327–337.
[41] T. T. Vu and H. D. Vo, Using multiple code representations to prioritize static analysis warnings (2022) 1–6.
[42] X. Hu, G. Li, X. Xia, D. Lo and Z. Jin, Deep code comment generation (2018) 200–20010.
[43] Z. Liu, X. Xia, C. Treude, D. Lo and S. Li, Automatic generation of pull request descriptions (2019) 176–188.
[44] S. Iyer, I. Konstas, A. Cheung and L. Zettlemoyer, Summarizing source code using a neural attention model (2016) 2073–2083.
[45] A. Mastropaolo, S. Scalabrino, N. Cooper, D. N. Palacio, D. Poshyvanyk, R. Oliveto and G. Bavota, Studying the usage of text-to-text transfer transformer to support code-related tasks (2021) 336–347.
[46] Y. Wan, Z. Zhao, M. Yang, G. Xu, H. Ying, J. Wu and P. S. Yu, Improving automatic source code summarization via deep reinforcement learning (2018) 397–407.
[47] L. Li, H. Feng, W. Zhuang, N. Meng and B. Ryder, Cclearner: A deep learning-based clone detection approach (2017) 249–260.
[48] P. Godefroid, H. Peleg and R. Singh, Learn&fuzz: Machine learning for input fuzzing (2017) 50–59.
[49] Y. Li, S. Wang, T. N. Nguyen and S. Van Nguyen, Improving bug detection via context-based code representation learning and attention-based neural networks, *Proceedings of the ACM on Programming Languages* **3**(OOPSLA) (2019) 1–30.
[50] N. Jiang, T. Lutellier and L. Tan, Cure: Code-aware neural machine translation for automatic program repair (2021) 1161–1173.
[51] Y. Ding, B. Ray, P. Devanbu and V. J. Hellendoorn, Patching as translation: the data and the metaphor (2020) 275–286.